

EE451 PHW5
Rafael Wang
rafaelwa@usc.edu
USC ID: 6189106477

For the entire PHW, the test platform used was the USC CARC Discovery cluster. The programs were run using a job script with configurations of `--nodes=1`, `--ntasks=1`, `--cpus-per-task=8`, `--mem-per-cpu=16GB`.

To run my code, after logging into CARC, first run:

```
module load legacy/CentOS7
module load gcc/9.2.0
module load nvidia-hpc-sdk
to load the CUDA module.
```

Then, run:

```
make
```

to generate all the executables.

Then, run:

```
sbatch job.sl
```

to run all the problems.

The outputs from CARC for all the problems can be found in `gpujob.out`.

The output from CARC for problem 1.1 is shown below:

```
===== Running p1_memory =====
time is 7273502.000000 ns
c[451][451]=2048
```

The output from CARC for problem 1.2 is shown below:

```
===== Running p2_memory =====
time is 5630445.000000 ns
c[451][451]=2048
```

The output from CARC for problem 2.1 is shown below:

```
===== Running p1_async =====
=== nStreams = 1 ===
time is 6793727.874756 ns
c[451][451]=208282624
=== nStreams = 4 ===
time is 6312799.930573 ns
c[451][451]=208282624
=== nStreams = 16 ===
time is 6183584.213257 ns
c[451][451]=208282624
```

The output from CARC for problem 2.2 is shown below:

```
===== Running p2_async =====  
=== nStreams = 1 ===  
time is 6802815.914154 ns  
c[451][451]=208282624  
=== nStreams = 4 ===  
time is 6302048.206329 ns  
c[451][451]=208282624  
=== nStreams = 16 ===  
time is 6194719.791412 ns  
c[451][451]=208282624
```

Table comparing execution times of all four implementations:

Implementation	Execution time (ns)
Global Memory	7273502.000000
Shared Memory Tiling	5630445.000000
Sequential Issue Loop (nStreams = 1)	6793727.874756
Sequential Issue Loop (nStreams = 4)	6312799.930573
Sequential Issue Loop (nStreams = 16)	6183584.213257
Parallel Issue Loop (nStreams = 1)	6802815.914154
Parallel Issue Loop (nStreams = 4)	6302048.20632
Parallel Issue Loop (nStreams = 16)	6194719.791412

For both the asynchronous versions, as the number of streams increased, the execution time improved. This is reasonable as it reflects greater overlap between memory copies and kernel execution. The improvement in execution time as the number of streams went from 1 to 4 was greater than when the number of streams went from 4 to 16, reflecting how execution time slowly plateaus.

I did not observe a significant speedup between the sequential and parallel issue loops. While the execution time was faster for the parallel issue loop using nStreams = 4, the sequential issue loop was faster using nStreams = 16. Factors such as run-to-run variance or too small of a matrix size could have prevented me from observing one being superior to the other.